



Themenübersicht

Modul 164

Überarbeitung, 28.02.2022, Gerd Gesell/ Thanam Pangri





Inhaltsverzeichnis

1	DBMS (Datenbank Management System)	4
1.1	Merkmale eines DBMS	4
1.2	Vorteile des Datenbankeinsatzes	6
1.3	Nachteile von Datenbanksystemen	6
1.4	Produkte	7
2	Anlegen einer Datenbank	9
2.1	Definition Character Set	10
3	Löschen in professionellen Datenbanken	12
4	Mehrfachbeziehungen	13
5	Generalisierung/Spezialisierung	14
6	Datensicherung von Datenbanken (Quelle: ionos.de)	15
6.1	Warum ist es wichtig, Datenbanken zu sichern?	15
6.2	Möglichkeiten zur Sicherung von Datenbanken	15
6.3	Backups erzeugen	16
6.4	Datenbanken müssen immer geschützt werden	17
7	Rekursion	18
7.1	einfache Hierarchie	19
7.2	Stücklistenproblem	19
8	identifying relationship vs. non-identifying relationship	21
9	Suchverfahren	22
10	Indextabellen	23
11	Tabellentypen und Transaktionen	24
11.1	Tabellentypen und Transaktionsmodelle	24
11.2	Transaktionen	25



11.3	ACID	32
12	Stored Procedures und Funktionen	34
13	Index	35

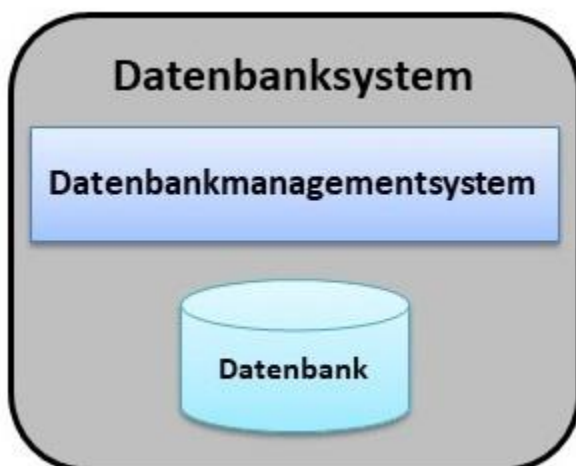


1 DBMS (Datenbank Management System)

Ein **Datenbanksystem (DBS)** ist ein System zur elektronischen Datenverwaltung. Die wesentliche Aufgabe eines DBS ist es, große Datenmengen effizient, widerspruchsfrei und dauerhaft zu speichern und benötigte Teilmengen in unterschiedlichen, bedarfsgerechten Darstellungsformen für Benutzer und Anwendungsprogramme bereitzustellen.

Ein DBS besteht aus zwei Teilen: der Verwaltungssoftware, genannt **Datenbankmanagementsystem (DBMS)** und der Menge der zu verwaltenden Daten, der eigentlichen **Datenbank (DB)**. Die Verwaltungssoftware organisiert intern die strukturierte Speicherung der Daten und kontrolliert alle lesenden und schreibenden Zugriffe auf die Datenbank. Zur Abfrage und Verwaltung der Daten bietet ein Datenbanksystem eine Datenbanksprache an.

Datenbanksysteme gibt es in verschiedenen Formen. Die Art und Weise, wie ein solches System Daten speichert und verwaltet, wird durch das Datenbankmodell festgelegt. Die bekannteste Form eines Datenbanksystems ist das **Relationale Datenbanksystem**.



1.1 Merkmale eines DBMS

Gemäss seiner Definition muss ein DBMS folgende Funktionalitäten bieten:

Integrierte Datenhaltung. Das DBMS ermöglicht die einheitliche Verwaltung aller von den Anwendungen benötigten Daten. Somit wird jedes logische Datenelement, wie beispielsweise der Name eines Kunden, an nur einer Stelle in der Datenbank gespeichert. Dabei muss ein DBMS die Möglichkeit bieten, eine Vielzahl komplexer Beziehungen zwischen den Daten zu definieren sowie zusammenhängende Daten schnell und effizient miteinander zu verknüpfen. In manchen Fällen kann eine kontrollierte Redundanz allerdings nützlich sein, um die Effizienz der Verarbeitung von Anfragen zu verbessern.

Sprache. Das DBMS stellt an seiner Schnittstelle eine Datenbanksprache (*query language*) für die folgenden Zwecke zur Verfügung:



- Datenanfrage (retrieval),
- Datenmanipulation (Data Manipulation Language, DML),
- Verwaltung der Datenbank (Data Definition Language, [DDL](#)),
- Berechtigungssteuerung (Data Control Language, DCL).

Da viele verschiedene Benutzer mit unterschiedlichen technischen Kenntnissen und Anforderungen auf eine Datenbank zugreifen, sollte ein DBMS eine Vielzahl von Benutzeroberflächen bereitstellen. Dazu zählen Anfragesprachen für gelegentliche Benutzer, Programmierschnittstellen und grafische Benutzeroberflächen (Graphical User Interface, GUI). Die Möglichkeit eines Webzugriffs ist heute eine Grundvoraussetzung für den Einsatz eines DBMS.

Katalog. Der Katalog (*data dictionary*) ermöglicht Zugriffe auf die Datenbeschreibungen der Datenbank, die auch als Metadaten bezeichnet werden.

DATA					DATA DICTIONARY (METADATA)		
employee_id	first_name	last_name	nin	dept_id	Column	Data Type	Description
44	Simon	Martinez	HH 45 09 73 D	1	employee_id	int	Primary key of a table
45	Thomas	Goldstein	SA 75 35 42 B	2	first_name	nvarchar(50)	Employee first name
46	Eugene	Comelsen	NE 22 63 82	2	last_name	nvarchar(50)	Employee last name
47	Andrew	Petculescu	XY 29 87 61 A	1	nin	nvarchar(15)	National Identification Number
48	Ruth	Stadick	MA 12 89 36 A	15	position	nvarchar(50)	Current position title, e.g. Secretary
49	Bary	Scardelis	AT 20 73 18	2	dept_id	int	Employee department. Ref: Departments
50	Sidney	Hunter	HW 12 94 21 C	6	gender	char(1)	M = Male, F = Female, Null = unknown
51	Jeffrey	Evans	LX 13 26 39 B	6	employment_start_date	date	Start date of employment in organization.
52	Doris	Berndt	YA 49 88 11 A	3	employment_end_date	date	Employment end date.
53	Diane	Eaton	BE 08 74 68 A	1			

Abbildung 1: data dictionary

Benutzersichten. Für unterschiedliche Klassen von Benutzern sind verschiedene Sichten (*views*) erforderlich, die bestimmten Ausschnitte aus dem Datenbestand beinhalten oder diesen in einer für die jeweilige Anwendung benötigten Weise strukturieren. Die Sichten sind im externen Schema der Datenbank definiert.

Konsistenzkontrolle. Die Konsistenzkontrolle, auch als Integritätssicherung bezeichnet, übernimmt die Gewährleistung der Korrektheit von Datenbankinhalten und der korrekten Ausführung von Änderungen. Ein korrekter Datenbankzustand wird durch benutzerdefinierte Integritätsbedingungen (*constraints*) im DBMS definiert, die während der Laufzeit der Anwendungen vom System kontrolliert werden. Daneben wird aber auch die physische Integrität sichergestellt, d. h. die Gewährleistung intakter Speicherstrukturen und Inhalte (Speicherkonsistenz).

Datenzugriffskontrolle. Durch die Festlegung von Regeln kann der unautorisierte Zugriff auf die in der Datenbank gespeicherten Daten verhindert werden (*access control*). Dabei kann es sich um personenbezogene Daten handeln, die datenschutzrechtlich relevant sind, oder um firmenspezifische Datenbestände. Rechte lassen sich auch auf Sichten definieren.

Transaktionen. Mehrere Datenbankänderungen, die logisch eine Einheit bilden, lassen sich zu Transaktionen zusammenfassen, die als Ganzes ausgeführt werden sollen (*Atomarität*) und deren Effekt bei Erfolg permanent in der Datenbank gespeichert werden soll (*Dauerhaftigkeit*).



Mehrbenutzerfähigkeit. Konkurrierende Transaktionen mehrerer Benutzer müssen synchronisiert werden, um gegenseitige Beeinflussung, z. B. bei Schreibkonflikten auf gemeinsam genutzten Daten, zu vermeiden (concurrency control). Dem Benutzer erscheinen die Daten so, als ob nur eine Anwendung darauf zugreift (Isolation).

Datensicherung. Das DBMS muss in der Lage sein, bei auftretenden Hard- oder Softwarefehlern wieder einen korrekten Datenbankzustand herzustellen (recovery).

1.2 Vorteile des Datenbankeinsatzes

Zusätzlich zu den genannten Merkmalen ergibt sich bei Einsatz eines Datenbanksystems eine Reihe von Vorteilen:

Nutzung von Standards. Der Einsatz einer Datenbank erleichtert Einführung und Umsetzung zentraler Standards in der Datenorganisation. Dies umfasst Namen und Formate von Datenelementen, Schlüssel, Fachbegriffe.

Effizienter Datenzugriff. Ein DBMS gebraucht eine Vielzahl komplizierter Techniken zum effizienten Speichern und Wiederauslesen (retrieval) großer Mengen von Daten.

Kürzere Softwareentwicklungszeiten. Ein DBMS bietet viele wichtige Funktionen, die allen Anwendungen, die auf eine Datenbank zugreifen wollen, gemeinsam ist. In Verbindung mit den angebotenen Datenbanksprachen wird somit eine schnellere Anwendungsentwicklung ermöglicht, da der Programmierer von vielen Routineaufgaben entlastet wird.

Hohe Flexibilität. Die Struktur der Datenbank kann bei sich ändernden Anforderungen ohne große Konsequenzen für die bestehenden Daten und die vorhandenen Anwendungen modifiziert werden (Datenunabhängigkeit)

Hohe Verfügbarkeit. Viele transaktionsintensive Anwendungen (z. B. Reservierungssysteme, Online-Banking) haben hohe Verfügbarkeitsanforderungen. Ein DBMS stellt die Datenbank allen Benutzern dank der Synchronisationseigenschaften **gleichzeitig** zur Verfügung. Änderungen werden nach Transaktionsende sofort sichtbar.

Große Wirtschaftlichkeit. Die durch den Einsatz eines DBMS erzwungene Zentralisierung in einem Unternehmen erlaubt die Investition in leistungstärkere Hardware, statt jede Abteilung mit einem eigenen (schwächeren) Rechner auszustatten. Somit reduziert der Einsatz eines DBMS die Betriebs- und Verwaltungskosten.

1.3 Nachteile von Datenbanksystemen

Trotz der Vorteile eines DBMS gibt es auch Situationen, in denen ein solches System unnötig hohe Zusatzkosten im Vergleich zur traditionellen Dateiverarbeitung mit sich bringen würde:

- Hohe Anfangsinvestitionen für Hardware und Datenbanksoftware.
- Ein DBMS ist Allzweck-Software, somit weniger effizient für spezialisierte Anwendungen.



- Bei konkurrierenden Anforderungen kann das Datenbanksystem nur für einen Teil der Anwendungsprogramme optimiert werden.
- Mehrkosten für die Bereitstellung von Datensicherheit, Mehrbenutzer- Synchronisation und Konsistenzkontrolle.
- Hochqualifiziertes Personal erforderlich, z. B. Datenbankdesigner, Datenbankadministrator.
- Verwundbarkeit durch Zentralisierung (Ausweg Verteilung).

Unter bestimmten Umständen ist der Einsatz regulärer Dateien sinnvoll:

- Ein gleichzeitiger Zugriff auf die Daten durch mehrere Benutzer ist nicht erforderlich.
- Für die Anwendung bestehen feste Echtzeitanforderungen, die von einem DBMS nicht ohne weiteres erfüllt werden können.
- Es handelt sich um Daten und Anwendungen, die einfach und wohldefiniert sind und keinen Änderungen unterliegen werden.

1.4 Produkte

DBMS	\$?	Hersteller	Modell/Charakteristik
Adabas	\$	Software AG	NF ² -Modell (nicht normalisiert)
Cache	\$	InterSystems	hierarchisch, "postrelational"
DB2	\$	IBM	objektrelational
Firebird		—	relational, basierend auf InterBase
IMS	\$	IBM	hierarchisch, Mainframe – DBMS
Informix	\$	IBM	objektrelational
InterBase	\$	Borland	relational
MS Access	\$	Microsoft	relational, Desktop – System
MS SQL Server	\$	Microsoft	objektrelational
MySQL		MySQL AB	relational
Oracle	\$	ORACLE	objektrelational
PostgreSQL		—	objektrelational, hervorgegangen aus Ingres und Postgres
Sybase ASE	\$	Sybase	relational



Versant	\$	Versant	objektorientiert
Visual FoxPro	\$	Microsoft	relational, Desktop – System
Teradata	\$	NCR Teradata	relationales Hochleistungs DBMS, speziell für Data Warehouses



2 Anlegen einer Datenbank

In MySQL ist «Schema» ein Synonym für «Datenbank». In einem DBS können Sie mehrere Schema's anlegen. Das macht auch Sinn, da man damit die Daten besser strukturieren und gruppieren kann und somit mehr Datenordnung ins DBS reinbringt.

Zusammenfassend kann man sagen: Ein DBS ist u.a. eine Sammlung aus Datenbanken und eine Datenbank ist eine Sammlung aus mehreren Tabellen.

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
    [create_option] ...

create_option: [DEFAULT] {
    CHARACTER SET [=] charset_name
    | COLLATE [=] collation_name
    | ENCRYPTION [=] {'Y' | 'N'}
}
```

Referenzdokumentation finden Sie hier: <https://dev.mysql.com/doc/refman/8.0/en/create-database.html>

Beispiel:

```
CREATE SCHEMA IF NOT EXISTS `DDL_DML_Uebungen`
DEFAULT CHARACTER SET utf8mb4;
```

Wichtig ist, dass Sie den Charakter Set gut überlegt festlegen. Beim Schema bzw. Datenbank kann er als Default Wert definiert werden. Es gibt aber die Möglichkeit, untergeordnet auf Tabellen- oder Spaltenebene zu definieren. Das bedeutet, dass der Charakter Set untergeordnet übersteuert wird.

```
CREATE TABLE `schuhe` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(45) CHARACTER SET latin1 COLLATE latin1_swedish_ci NOT NULL,
  `groesse` varchar(4) DEFAULT NULL,
  `preis` float NOT NULL,
  `farbe` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```



2.1 Definition Character Set

Hier ein Auszug aus dem Buch «Grundkurs_mySql_MariaDb», Seite 64.

Was ist ein Zeichensatz? Auf dem Computer werden Buchstaben, Ziffern, Satz- und Sonderzeichen durch Zahlen kodiert. So ist beispielsweise der Buchstabe A im ASCII¹³ die Zahl 0x41 und a die Zahl 0x61. Da für die Kodierung des ASCII nur ein Byte (= 8 Bit) zur Verfügung steht, können nur $2^8 = 256$ verschiedene Zahlen zur Kodierung verwendet werden.

Die ersten 128 sind im Wesentlichen die Steuerzeichen (wie z.B. der Zeilenumbruch), das Leerzeichen, die lateinischen Buchstaben, die Ziffern 0 – 9, Satz- und einfache Sonderzeichen. Die restlichen 128 wurden mehr oder weniger willkürlich dazu verwendet, Umlaute oder andere sprachspezifische Sonderzeichen abzubilden.

Und hier fing das Unglück an. Fast jeder Computer- oder Betriebssystemhersteller hat da sein eigenes Süppchen gekocht. So ist beispielsweise das Zeichen Ü im Zeichensatz ISO/IEC 8859-1 mit der Zahl 0xDC kodiert und in Codepage 850 mit 0x9A. Wird nun ein Text unter Windows erfasst, wird das Ü als 0xDC in die Datei geschrieben. Öffnet man nun diese Datei mit einem COMMAND-Editor wie EDIT, so erscheint aber ein anderes Zeichen und umgekehrt.

Dieses Problem und die Beschränkung auf 256 Zeichen, was die Darstellung z.B. ostasiatischer Schriften unmöglich macht, haben dazu geführt, dass man eine neue, leicht erweiterbare Kodierung von Schriftzeichen baute. Unicode ward geboren! Unicode selbst liegt in verschiedenen Formatierungen vor. Derzeit gerne verwendet werden utf8, utf16 und utf32.



Für die meisten Anwendungen in Deutschland sind die Zeichensätze `latin1`, `cp850` und `utf8` ausreichend. Welchen Zeichensatz Sie tatsächlich brauchen, ist genau zu untersuchen und hängt in der Regel von der Datenquelle ab¹⁴.

Stammen Ihre Daten aus einer älteren Quelle und sind ggf. über ein `COMMAND`-Terminal eingegeben worden, ist `cp850` vermutlich richtig¹⁵. Neue Anwendungen sollten von Anfang an `utf8` verwenden. Diese Kodierung ist recht stabil bezüglich der Sonderzeichen und Umlaute und macht auch die Verwaltung von anderssprachlichen Zeichen einfach möglich. Ein weiterer Vorteil ist der im Verhältnis zu `utf16` und `utf32` geringere Speicherverbrauch.

Microsoft verwendet in seiner .NET-Umgebung `utf16`. Falls Sie also eine Anwendung bauen, die mit ADO/.NET arbeiten soll, ist die Verwendung von `utf16` für die Datenbank trotz ihrer Speicherplatzverschwendung sinnvoll.

So sieht unser Anlegen einer Datenbank bisher aus:

- 1 `DROP SCHEMA IF EXISTS oshop;`
- 2 `CREATE SCHEMA oshop`
- 3 `DEFAULT CHARACTER SET utf8;`



3 Löschen in professionellen Datenbanken

Die Standardoperationen, die über SQL an Datenbankbeständen von den Anwendern ausgeführt werden, sind Daten erfassen, verändern, abfragen und löschen. In fast allen professionellen Applikationen ist die Löschfunktion – also der SQL-Befehl delete – jedoch tabu. Damit wäre ein Informationsverlust verbunden, der aus verschiedenen Gründen unerwünscht ist.

Ein typischer Denkfehler von Datenbankneulingen ist es zum Beispiel, einen austretenden Mitarbeiter aus der Personaltabelle zu löschen. Wenn ich diesen Datensatz aber wirklich entferne, gehen mir sämtliche Beziehungen verloren, die damit in Verbindung stehen. Ich kann also nicht mehr nachvollziehen, welche Aktionen dieser Mitarbeiter früher gemacht hat. Innerhalb von Banken werden zum Beispiel die Zugriffe auf Kontodaten genau protokolliert. Wenn der austretende Mitarbeiter also gelöscht würde, wären sämtliche Aktivitäten von ihm nicht mehr nachvollziehbar. Das könnte zum Teil sogar juristisch Probleme geben.

Die Lösung im Falle der Mitarbeiterdaten wäre also, dass ich keine Daten lösche, sondern meinen Informationsbestand um die Austrittsdaten erweitere und den Mitarbeiterdatensatz eventuell als inaktiv markiere, was aber aufgrund der Austrittsdaten genau genommen redundant ist.

Auch zeitliche Abläufe werden nicht durch überschreiben (update) bestehender Einträge abgebildet. Ein Gegenstand, der mehrmals hintereinander an verschiedene Personen verliehen wird, hat nicht nur einen Fremdschlüssel, der auf die jeweils aktuelle Person zeigt, sondern ich benötige weitere Tabellen, um die zeitliche Dynamik statisch abzubilden. Schliesslich sollen von der Datenbank Informationen und Auswertungen auch aus der Vergangenheit gemacht werden (z. B. wieviel Prozent der Zeit war der Artikel x ausgeliehen?).

Betrachten wir als nächstes Beispiel ein Kassensystem. Wenn ein Einkauf getätigt und erfasst worden ist, merkt der Kunde, dass er zu wenig Bargeld dabei hat und möchte auf den Kauf verzichten. Wenn das Datenbanksystem jetzt ein einfaches Löschen der zugehörigen Datensätze erlaubt, wäre das gleiche Verhalten auch missbräuchlich verwendbar. Es können ein paar Datensätze gelöscht werden und schon wäre überzähliges Bargeld in der Kasse und jeden Abend Party in der Verkaufsabteilung. Andererseits kann ich den Kunden auch nicht zum Kauf verpflichten, nur weil meine Datenbank keine Korrekturmöglichkeit hat. Dieses Problem ist aber über Stornierungen möglich. Dabei kann ich je nach Informationsstruktur spezielle Tabellen für die Stornierungsbuchungen haben oder ich speichere sie in den gleichen Tabellen wie die Buchungen ab, aber mit zusätzlichen Informationen für die Zeit, Stornierungsgrund usw.

Wenn die Datenbank mit referentieller Integrität erstellt wurde, werden die Beziehungen vom DBMS verwaltet und überwacht. In diesem Fall kann ich die Daten schon aus technischen Gründen nicht löschen, es sein denn ich lösche auch die in Beziehung stehenden Datensätze. Stellen Sie sich aber ein System wie Wikipedia vor. Der Benutzer hat Einträge und Veränderungen an diversen Orten vorgenommen. Diese Veränderungen sind dann von anderen Benutzern auch wieder verändert worden. Da in Wikisystemen alle Aktivitäten historisiert werden, kann ich aus dieser Veränderungshistorie nicht einfach Informationen löschen. Aus diesem Grund werden auf Wikipedia Benutzernamen auch dann nicht gelöscht, wenn sie zur Kategorie „bäh“ gehören. Sie werden bestenfalls gesperrt, damit unter diesen Namen keine weiteren Aktivitäten erfolgen können.



4 Mehrfachbeziehungen

Bei Mehrfachbeziehungen haben zwei Tabellen mehrere Beziehungen, die unabhängig voneinander sind und jeweils einen anderen Sachverhalt repräsentieren. Um die verschiedenen Beziehung eindeutig zu kennzeichnen, müssen sie möglichst aussagekräftig beschriftet werden. Die Kardinalitäten können natürlich verschieden sein, da die Beziehungen voneinander unabhängig sind. In den Beispielen unten gibt es drei unabhängige Beziehungen zwischen den Entitätstypen *tbl_Fahrten* und *tbl_Orte*. Eine dieser Beziehungen ist die Auflistung der Orte, die bei einer Tour angefahren werden können. Dies ist eine mc:mc-Beziehung und benötigt daher eine Transformationstabelle.

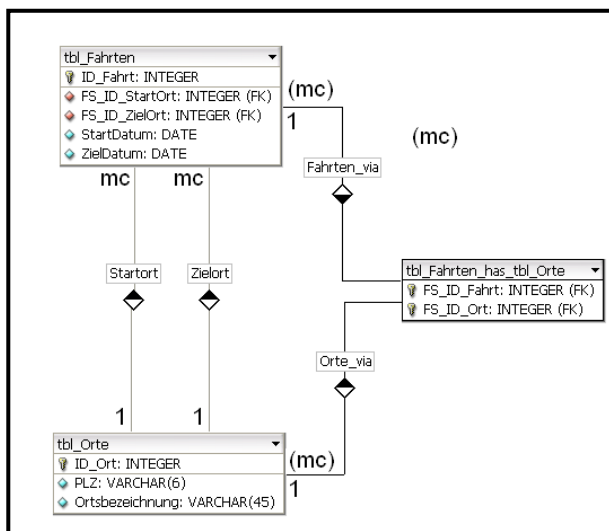


Abbildung 2: Darstellung mit DBDesigner4 (Kardinalitäten wurden nachträglich eingetragen)

Bei der Abbildung 3 ist der gleiche Sachverhalt dargestellt. Nur unterscheidet sich die Darstellung in MS-Access, da es nicht in der Lage ist mehr als eine Beziehung zwischen 2 Tabellen grafisch darzustellen. Das Problem wird gelöst, indem eine Tabelle „geclont“ wird (zu erkennen an der Bezeichnungserweiterung _1). Physikalisch sind *tbl_Orte* und *tbl_Orte_1* jedoch die gleiche Tabelle.

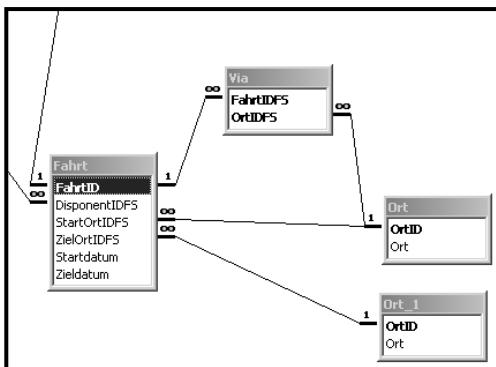


Abbildung 3: Darstellung im Beziehungsfenster von MS-Access



5 Generalisierung/Spezialisierung

Die Datenbankmodellierung, die wir hier betrachten beruht auf dem Attributkonzept. Wir definieren Attribute mit Attributsausprägungen und ordnen diese den Entitätstypen zu. Problematisch wird es, wenn jetzt mehrere Entitätstypen viele Attribute gemeinsam haben und einige nicht. Jetzt könnte Redundanz entstehen, wenn ein konkretes Objekt aus der realen Welt durch mehrere Entitätstypen beschrieben wird. In unserem konkreten Beispiel wären das Mitarbeiter, die auch als Kunden auftreten oder Fahrer, die auch als Disponenten arbeiten. In Anlehnung an [ZEHNDER, Carl August (1989): Informationssysteme und Datenbanken] dürfen „lokale Attribute“ nur einmal in einer Datenbank auftreten, da sonst Redundanz vorliegt. Die Lösung des Problems besteht darin, dass die gemeinsamen Attribute in einem allgemeinen Entitätstypen zusammengefasst werden (Generalisierung). Die nicht gemeinsamen Attribute verbleiben in den Entitätstypen (Spezialisierung). Um einen Informationsverlust zu vermeiden, müssen die Datensätze der spezialisierten Tabellen per Fremdschlüssel auf die generalisierten Tabellen verweisen. Diese Beziehung wird auch als „is-a“-Beziehung bezeichnet: eine Person *ist ein* Fahrer.

Diese Beziehungsform ist auch in der objektorientierten Modellierung bekannt und wird dort mittels Vererbung gelöst.

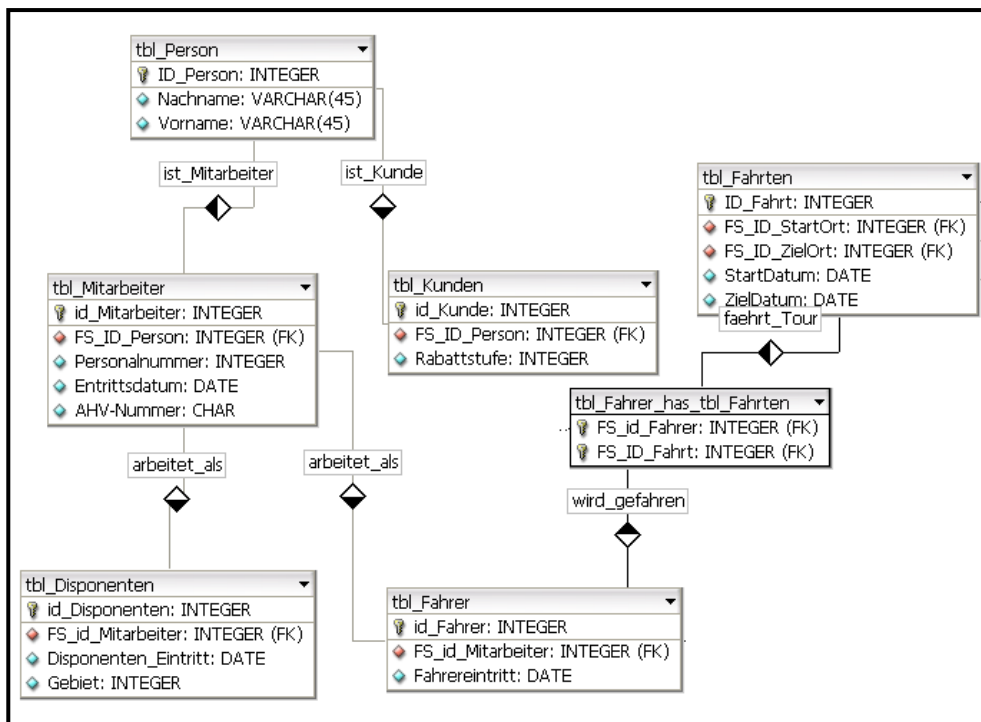


Abbildung 4: Beispiel für Generalisierung/Spezialisierung



6 Datensicherung von Datenbanken (Quelle: ionos.de)

Folgende Situation wird vielen Computernutzern bekannt vorkommen: Aus Versehen wurde eine wertvolle Datei gelöscht, von der es leider keine Sicherungskopie gibt. Die Datei ist somit dauerhaft verloren. So ein Verlust ist ärgerlich. Sind jedoch nicht nur einzelne Daten betroffen, sondern ganze Datenmengen, kann das – insbesondere für Unternehmen – verheerend sein. Im Unternehmenssektor, aber auch für viele Privatanwender ist es daher unabdingbar, die eigenen Daten zu sichern.

6.1 Warum ist es wichtig, Datenbanken zu sichern?

Datenbanksysteme spielen sowohl beim Webhosting wie auch als Bestandteil von Unternehmenssoftware eine große Rolle. Denn von der Verfügbarkeit und Vollständigkeit der gespeicherten Daten hängen die Funktionalität der Website bzw. die Aktionsfähigkeit des Unternehmens ab.

Website-Projekte, die auf Datenbanken zugreifen, entnehmen selbigen z.B. mithilfe verschiedener Skript-Sprachen alle Informationen, die für das korrekte Anzeigen der Seite notwendig sind. Auch die IT-Infrastruktur eines Unternehmens zieht ihre Informationen in der Regel aus den zugrundeliegenden Datenbanken. Gleichzeitig findet der **Datenaustausch** ebenso in umgekehrter Richtung statt, indem Benutzer Daten in der Datenbank ablegen bzw. speichern. So beinhalten Datenbanksysteme nicht selten Personal- und Finanzinformationen oder sensible Kundendaten. Entsprechend schwer wiegt ein Ausfall der Datenbank oder gar ein Datenverlust: Die Website präsentiert die Inhalte nicht mehr vollständig oder ist komplett offline, Anwendungen funktionieren nicht mehr und die Kundendaten sind lückenhaft oder fehlen schlimmstenfalls komplett. Das sorgt für zusätzliche Arbeit beim Betroffenen und Irritation beim Kunden, die möglicherweise einen dauerhaften Vertrauensverlust zur Folge hat.

Die Ursache für einen Datenverlust ist in den meisten Fällen kein Angriff von außen, sondern technisches Versagen der Hardware oder schlicht ein Benutzerfehler. Davor kann auch die beste Sicherheitssoftware nicht schützen. Damit der Datenverlust nicht irreversibel ist, ist eine Datensicherung erforderlich.

6.2 Möglichkeiten zur Sicherung von Datenbanken

Um einem Datenverlust vorzubeugen, sollte man Sicherheitskopien der Datenbanken auf externen Speichermedien erstellen. Mit diesen Kopien, die auch Backups genannt werden, kann der Zustand der Datenbank zum Zeitpunkt der Datensicherung wiederhergestellt werden.

Hierbei ist zunächst zwischen Online- und Offline-Backups zu unterscheiden: Online-Backups werden erzeugt, ohne dass die Datenbank heruntergefahren werden muss. Während des Sicherungsprozesses nimmt die Datenbank vorgenommene Änderungen in einen separaten Bereich auf und fügt sie erst im Anschluss an die Sicherung in die entsprechende Datei ein. Führt man die Datenbank für die Zeit der Sicherung herunter, spricht man von einem Offline-Backup. Dieses Verfahren zur Datensicherung hat zwar den Vorteil, dass es relativ unkompliziert durchzuführen ist, bringt jedoch auch den Nachteil mit sich, dass die Anwendungen oder die Websites während des Backups nicht verfügbar sind. Daher sollte ein Offline-Backup möglichst in der Nacht bzw. zu Zeiten geringen Datenverkehrs durchgeführt werden.



Zusätzlich zur Unterteilung in Online- und Offline-Backup unterscheidet man beim Überspielen der Daten auf ein Sicherheitsmedium folgende drei Arten der Sicherung:

- **Voll-Backup:** Wie der Name schon verrät, werden bei dieser Art der Datensicherung immer sämtliche Daten überspielt. Das hat zur Folge, dass pro Backup zwar alle Dateien vorliegen, der benötigte Speicherplatz bei häufiger Sicherung jedoch sehr hoch ist. Für die Wiederherstellung wird nur das jeweilige Voll-Backup benötigt.
- **Differentielles Backup:** Bei einem differentiellen Backup wird zunächst ein Voll-Backup erstellt. Die folgenden Sicherungen unterscheiden sich von der ersten insofern, dass lediglich Dateien gesichert werden, die sich geändert haben oder neu hinzugekommen sind. Im Gegensatz zum Voll-Backup wird so **Speicherplatz** gespart. Allerdings werden geänderte und neue Dateien – bis zum nächsten Voll-Backup – bei jedem differentiellen Backup erneut kopiert. Die Recovery gelingt nur mit dem letzten Voll-Backup inklusive gewünschtem differentiellen Backup.
- **Inkrementelles Backup:** Im Anschluss an ein komplettes Backup werden bei der inkrementellen Datensicherung nur die Dateien kopiert, die sich seit der letzten Sicherung geändert haben oder neu hinzugekommen sind. Im Unterschied zu der differentiellen Methode bezieht sich ein inkrementelles Backup immer auf das vorherige (sowohl Voll-Backup als auch inkrementelles Backup). So wird jede Datei nur einmal gesichert und dadurch der Speicherplatz geschont. Um die gewünschten Dateien wiederherzustellen, werden allerdings alle Sicherungen vom Voll-Backup bis zum gewünschten Stand benötigt.

Es gibt also einige Optionen, Datenbanksysteme wie SQL-Datenbanken oder [Microsoft Access](#) zu sichern. Welche Sicherungsmethode jeweils am besten geeignet ist, hängt vom Anforderungsprofil des Users bzw. Unternehmens ab. Selten durchgeführte Sicherungen zum Sparen von Speicherplatz sollten aber niemals einen Lösungsweg darstellen. Externe Speichermedien wie z. B. Festplatten sollten zudem sicher und in einem separaten Bereich aufbewahrt werden, wo sie gegen Diebstahl, aber auch Brandschäden geschützt sind. Zusätzlich sollten die gesicherten Daten auch verschlüsselt werden, damit sie im Falle eines Diebstahls nicht von Dritten genutzt werden können.

6.3 Backups erzeugen

Hat man sich für eine Backup-Lösung entschieden, besteht der nächste Schritt darin, sich für eine Durchführungsmethode zu entscheiden. Es gibt verschiedene Möglichkeiten und Tools, um die Sicherung von Datenbanken, beispielsweise einer SQL-Datenbank, in die Wege zu leiten. Die folgende Auflistung erläutert einige davon:

- **MySQLDump:** Wer über einen Shell-Zugang verfügt, kann mit der integrierten Backup-Funktion von MySQL und dem Befehl „mysqldump“ arbeiten. Allerdings erlauben nicht alle Hosts den Zugriff auf diese Funktion, die im Übrigen die schnellste Backup-Durchführung ermöglicht.
- **phpMyAdmin:** Mit dieser Administrations-Plattform für SQL-Datenbanken können Benutzer die gewünschte Datenbank einfach im gewünschten Format, z. B. SQL, exportieren. Allerdings kann es dazu kommen, dass das **PHP-Script** bei zu großen Datenbanken vom Server abgebrochen wird. Zudem funktioniert nur die Einspielung von Backups mit einer Größe von bis zu 2 MB.
- **BigDump:** Das Tool BigDump bietet die perfekte Ergänzung zu phpMyAdmin, da es beliebig große Backups wieder einspielen kann. Eine eigene Sicherungs-Funktion bietet es allerdings nicht.



- HeidiSQL: Die Backup-Lösung für Windows-Systeme basiert nicht auf PHP und hat daher keine Probleme mit großen Backups. Dem Tool, das ansonsten phpMyAdmin sehr ähnlich ist, fehlt allerdings die Möglichkeit zur Automatisierung des Sicherungsprozesses.

6.4 Datenbanken müssen immer geschützt werden

Die in den Datenbanken gespeicherten Files haben häufig eine hohe Bedeutung für den reibungslosen Ablauf in Betrieben oder die korrekte Anzeige von Websites. Webserver greifen auf die enthaltenen Informationen zu, um die gehostete Website fachgerecht darstellen zu können, und auch die Funktionalität von Anwendungen im Netzwerk ist oftmals direkt mit einer Datenbank verbunden. Datenbanken bilden zudem auch den Speicherort sensibler Daten wie Adressen, Kontoinformationen oder Telefonnummern.

Aufgrund ihrer wichtigen Rolle sollte man die **Datenbanksysteme** unbedingt mit entsprechenden Sicherheitsmaßnahmen schützen. Während die Daten auf der einen Seite vor Angriffen von außen geschützt werden müssen, droht auf der anderen Seite nämlich auch der Verlust durch interne Probleme wie z. B. Hardware-Defekte oder Benutzerfehler. Regelmäßige Backups beugen einem Datenverlust vor und garantieren dadurch eine langfristige Datensicherheit.



7 Rekursion

Bis jetzt haben wir Beziehungen zwischen zwei verschiedenen Entitätstypen betrachtet. Es gibt aber auch die Möglichkeit, dass an einer Assoziation nur ein einziger Entitätstyp beteiligt ist. Konkret bedeutet das, dass ein Datensatz dieser Tabelle mit einem anderen Datensatz aus der gleichen Tabelle in Beziehung steht. Eine reine Hierarchie können wir innerhalb der gleichen Tabelle erledigen. Sie erhält also einen Fremdschlüssel, der auf den Identifikationsschlüssel der eigenen Tabelle verweist. Das klassische Beispiel hierfür ist eine Firmenorganisation, bei der jede Person genau eine andere Person als Vorgesetzten hat. Da die in der Hierarchie höchste Person natürlich keinen Vorgesetzten mehr haben kann, ist es eine c:mc Beziehung. Würden wir dieses Fremdschlüsselattribut auf „not null“ setzen (im Access: „Eingabe erforderlich“), könnten wir die „höchste Person nicht erfassen.

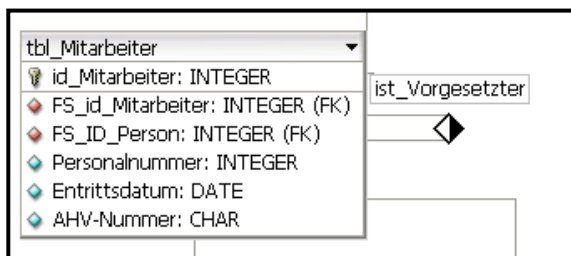


Abbildung 5: Darstellung einer einfachen Hierarchie in der Workbench

Mit dieser einfachen Variante kommen wir aber nicht weiter, wenn aus der klaren Hierarchie (einmal-Rekursion – nur ein Vorgesetzter ist möglich) eine Netzwerkstruktur wird. In diesem konkreten Beispiel bedeutet das, dass eine Person auch mehrerer Vorgesetzte haben kann. Dann handelt es sich um eine mc:mc Beziehung und wir benötigen eine Transformationstabelle. Speziell daran ist, dass die beiden Fremdschlüssel der Transformationstabelle jeweils auf einen Identifikationsschlüssel der gleichen Tabelle zeigen, aber in unterschiedlichen Rollen. Im Beispiel sind das die Rollen „ist Vorgesetzter von“ und „ist_Mitarbeiter_von“.

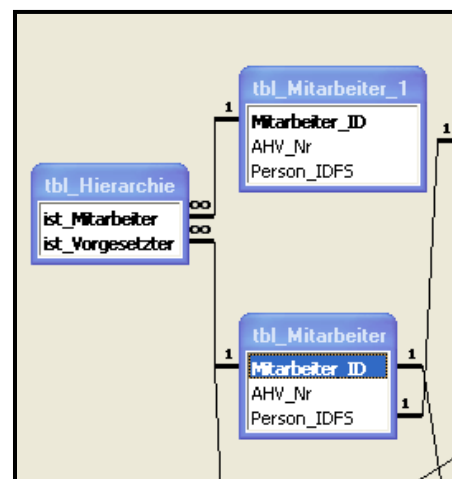
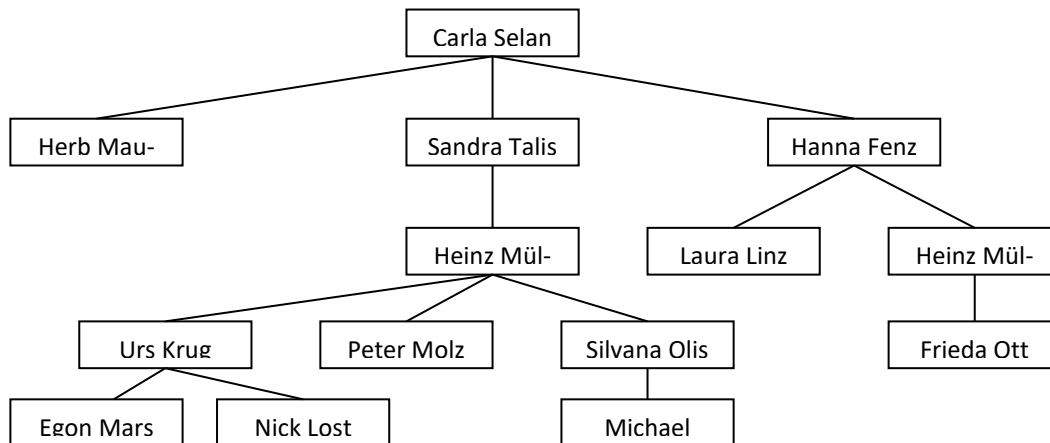


Abbildung 6: MS-Access-Darstellung einer netzwerkförmigen Struktur



7.1 einfache Hierarchie



Einfache Hierarchien können mit Rekursionen innerhalb der gleichen Tabelle abgebildet werden. Solange jedes Element nur maximal ein Oberelement hat, reicht ein Fremdschlüssel, der auf die Identifikationsschlüssel der eigenen Tabelle verweist. Lediglich das Wurzelement oder Topelement hat einen NULL-Wert, da die Hierarchie ja irgendwo enden (oder anfangen) muss. Es handelt sich hierbei um eine c:mc Beziehung.

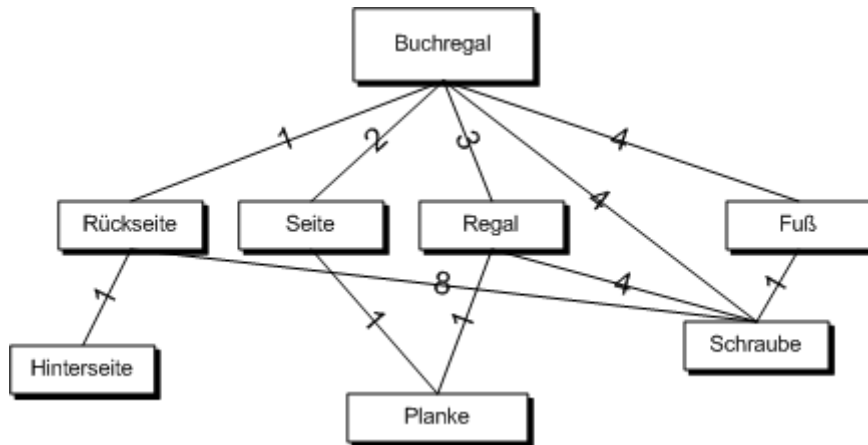
Wenn aber mehrere Oberelemente zugelassen sind (z. B. mehrere Projektleiter), dann handelt es sich um eine mc:mc-Beziehung und es wird eine Transformationstabelle benötigt, die für jeden Beziehungsgraphen einen Datensatz enthält, der angibt welches Oberelement zu welchem Unterelement gehört.

7.2 Stücklistenproblem

Eine klassische Anwendung der Rekursion in der Datenmodellierung ist die Stücklistenproblematik. Es gibt eine Tabelle mit Produkten und deren Eigenschaften. Jetzt kann sich aber auch ein Produkt aus mehreren anderen Produkten zusammensetzen (modulare Produkte). Beides – die zusammengesetzten Produkte (Aggregate) sowie die einzelnen Bestandteile – befindet sich in unserer Produkte- oder Artikeltabelle und wird auch einzeln verkauft. Das Problem besteht nun darin eine Liste mit Einzelteilen zu erzeugen, die alle Artikel auf elementarer Ebene enthält (die also nicht mehr aus mehreren Produktteilen bestehen).

Ein gutes Beispiel – sogar mit SQL-Code – finden Sie hier:

http://www.ianywhere.com/developer/product_manuals/sqlanywhere/1001/de/html/dbugde10/ug-parts-explosion-cte-sqlug.html



Das Beispiel beschreibt die Artikel einer Elementmöbelfabrik.

Lösung mit Rekursion: Da es sich im Modell ja nicht um eine einfache Hierarchie handelt (c:mc), reicht die Erstellung eines Fremdschlüsselattributs in der Produktetabelle nicht aus. Neben der Produktetabelle benötigen wir eine weitere Tabelle, die die Zusammensetzungen enthält. Also welche Komponente (Einzelteil oder Aggregat) fließt in welches Aggregat und mit welcher Menge ein?



8 identifying relationship vs. non-identifying relationship

Zur Unterscheidung der identifying relationship von der non-identifying relationship gibt es extrem viel Schrott im Internet. Eine kurze einfache Erklärung gibt es hier: <http://www.datenbank-grundlagen.de/beziehungen-datenbanken.html>

Bei der identifying-Beziehung ist der Fremdschlüssel Bestandteil des Indentifikationsschlüssels. Die ID ist also eine aus mehreren (mindestens 2) Attributen zusammengesetzte Schlüsselkombination und der Datensatz der referenzierenden Tabelle („unten“) identifiziert sich teilweise durch den Datensatz, auf den er verweist. Ein Raum kann in der ID also unter anderem den Fremdschlüssel auf das Gebäude haben. Eine non-identifying Beziehung wäre hier nicht so sinnvoll, denn darin könnte ich jederzeit den Fremdschlüsselwert ändern und den Raum einem anderen Gebäude zuordnen.

Die Beziehung Mitarbeiter/Abteilung ist da ein besseres Beispiel für eine non-identifying-Beziehung. Der Fremdschlüssel auf den Arbeitgeber gehört nicht zur Identität einer Person, zumal der Inhalt – also die Firma, auf die referenziert wird – jederzeit wechseln kann.



9 Suchverfahren

Um in grossen unsortierten Listen einen gewünschten Wert zu finden, müssen wir die lineare oder sequentielle Suche anwenden. Es bleibt uns nichts anderes übrig, als die gesamte Liste bis zum erfolgreichen Treffer oder gar bis zum Ende zu durchsuchen, wenn der Wert überhaupt nicht vorhanden ist. Aus diesem Grund ist es einfacher, wenn wir Listen – oder in Datenbanken dann Attribute – sortieren, um schnellere Verfahren anwenden zu können.

Bei der binären Suche oder der logarithmischen Suche wird eine sortierte Liste vorausgesetzt. Damit halbiert sich bei jedem Zugriff der Suchraum. Wir benötigen bei N Elementen $\log_2 N$ Zugriffe. Wenn unser Suchraum aus 16 ($=2^4$) Werten besteht, benötigen wir also maximal 4 Zugriffe (vorherige „Glückstreffer“ ausgenommen). Mit nur einem Zugriff mehr, können wir bis zu 32 Werte abdecken. Allerdings benötigen wir auch für 17 Werte maximal 5 Zugriffe. Für dieses Suchverfahren wird auch noch der Begriff „logarithmische Suche“ verwendet.

Eine Erweiterung der binären Suche ist die Interpolationssuche. Bei der binären Suche wird der Suchraum jedes Mal genau in der Mitte geteilt. Die Interpolationssuche berücksichtigt noch die Grösse des gesuchten Wertes in Bezug auf den Suchraum. Es vergleicht also den grössten und den kleinsten Wert, geht von einer gleichmässigen Verteilung der Daten aus und teilt den Suchraum jetzt im Verhältnis zu dem Suchwert. Konkret greift es bei einem sehr hohen Wert bereits im „wahrscheinlicheren“ hinteren Bereich des Suchraumes zu. Jeder erfolgreiche Zugriff schränkt den Suchraum also um mehr als die Hälfte ein. Mit der zusätzlichen Berechnung des Zugriffselementes geht aber ebenfalls Rechenzeit verloren. Daher ist die Interpolationssuche nur bei grossen Suchräumen effizienter.

Ein leicht verständliches Zahlenbeispiel finden Sie übrigens bei Wikipedia:

<http://de.wikipedia.org/wiki/Interpolationssuche>

Die Beschreibung rechts ist aus dem Buch Informationssysteme und Datenbanken; ISBN3 7281 2019 7 von Carl August Zehnder; vdf Hochschulverlag Zürich 6. Auflage Seite 224

Nun brauchen wir noch ein Suchverfahren, um die richtige Seite im Buch, bzw. den richtigen Eintrag auf der Seite *möglichst schnell* finden zu können. Da die Datei sortiert ist, können wir auf beiden Stufen (richtige Seite und richtiger Eintrag) das sog. *binäre Suchverfahren* (auch „Intervallschachtelung“ genannt) einsetzen (Fig.5.3).

	1. Schritt	2. Schritt	3. Schritt	Resultat
Ausgangsdatei: (sortiert)	02 17 75	vordere Hälfte	02 17 75	
mittl. Datensatz >	77	hintere Hälfte	75 77	75
	78 82 85			

Figur 5.3: Das binäre Suchverfahren („Intervallschachtelung“) braucht für $N=7$ drei Schritte; gesucht ist der Datensatz mit Schlüssel '75'.

Das binäre Suchverfahren funktioniert im wesentlichen für eine geordnete Datei von N Datensätzen wie folgt:

Jeder Arbeitsschritt halbiert die zu untersuchende Datei (Fig.5.3). Geprüft wird dazu einzig der mittlere Datensatz. Liegt der Suchschlüssel vor (bzw. hinter) dem Schlüssel des mittleren Datensatzes, arbeitet man im nächsten Arbeitsschritt nur noch mit der vorderen (bzw. hinteren) Hälfte weiter. Das Halbieren erfolgt solange, bis nur noch der gesuchte Datensatz übrig bleibt.

Es ist leicht zu sehen, dass dieses Verfahren nur wenige Arbeitsschritte braucht; bei 1000 vorsortierten Datensätzen lässt sich das gesuchte Element in jedem Fall mit 10 Halbierungen bestimmen (auf $500 - 250 - 125 - 63 - 32 - 16 - 8 - 4 - 2 - 1$); bei einer Million Datensätzen mit 20 Halbierungen. Es gilt folgende logarithmische Regel:

Def.: Das *binäre Suchen* benötigt bei N Elementen $\log_2 N$ Arbeitsschritte.



10 Indextabellen

Wenn wir grosse Datenbestände haben, ist es sinnvoll, die Attribute zu sortieren, wenn häufiger danach gesucht wird. Eine Tabelle kann aber logischerweise nur nach einem Attribut sortiert werden. Eine schlechte Lösung wäre es jetzt die ganze Tabelle zu kopieren und in der 2. Version nach einem anderen Kriterium zu sortieren. Die bessere Variante sind Indextabellen. Diese sind eine Reduktion der Haupttabelle auf das Schlüsselattribut und das Indexattribut und sind nach dem Indexattribut sortiert. Bei einem Suchzugriff erfolgt der erste Suchvorgang in der Indextabelle und dann mit dem gefundenen Schlüssel der zweite Suchvorgang in der Haupttabelle. Damit sind SELECT-Operationen immer noch deutlich schneller, als eine Tabelle mit mehrere Millionen Datensätzen sequenziell zu durchsuchen.

Ein paar Nachteile bleiben – und deswegen ist es nicht ratsam einfach alle Attribute aus Prinzip zu indexieren:

- auch die Indextabellen benötigen Platz,
- DELETE- und INSERT-Operationen und UPDATE-Operationen auf Indexattribute benötigen mehr Ressourcen, da auch die Indextabellen angepasst werden müssen.

Die Indextabellen werden automatisch vom DBMS verwaltet. Allerdings können Sie die Anzahl und die Grösse der vorhandenen Indices anzeigen lassen.

Für den Primärschlüssel und für Attribute, die auf unique gesetzt werden, wird automatisch eine Indextabelle erzeugt. Jeder Neueintrag muss auf Eindeutigkeit überprüft werden. Das entspricht einem SELECT mit dem Eingabewert auf das entsprechende Attribut und wird im positiven Fall eine leere Menge zurückgeben (der Wert ist erlaubt, da bis jetzt eindeutig) beziehungsweise im negativen Fall ein Ergebnis anzeigen (der Wert muss abgelehnt werden, da schon vorhanden).

Wenn eine Eindeutigkeit über mehrere Attribute erreicht werden soll, wird dies ebenfalls über eine Indextabelle realisiert. Sie können einen sogenannten „unique-Index“ unter Angabe aller notwendigen Attribute definieren. Damit ist gewährleistet, dass Werte innerhalb einzelner Attribute mehrfach vorkommen dürfen, aber die Kombination konkreter Werte über die ausgewählten Attribute nur einmal.

Der SQL-Befehl für die Indexierung lautet:

```
CREATE INDEX Indexname ON Tabellename (Spaltenname(n)) bzw.  
DROP INDEX Indexname
```

Die Indexbefehle sind zwar nicht im SQL-Standard enthalten, werden aber in der obigen Form von den meisten SQL-Datenbanksystemen erkannt.



11 Tabellentypen und Transaktionen

Die Entwickler von MySQL haben sich lange geweigert Mechanismen einzubauen, die nur dem Komfort der Entwickler dienen und sich auch auf andere Weise lösen liessen. Dazu gehört der Einbau der referentiellen Integrität genauso wie die Möglichkeit Transaktionen zu verwenden. Das macht den Standardtabellentyp von MySQL- zwar schnell und schlank, ist für einige Anwendungen aber nicht optimal. Seit Mai 2000 ist der Tabellentyp BDB (Berkley Database) integriert, der auch Transaktionen unterstützt. Mittlerweile ist das Repertoire noch um InnoDB und Gemini erweitert worden, die ebenfalls transaktionstauglich sind. Damit sind jetzt auch Techniken möglich, die jedes professionelle relationale Datenbanksystem einfach integriert haben muss.

11.1 Tabellentypen und Transaktionsmodelle

Ob Transaktionen und referentielle Integrität verwendet werden können, hängt also vom verwendeten Tabellentyp ab. Der ursprüngliche Tabellentyp von MySQL ist MyISAM und kennt keine Transaktionsunterstützung. Dazu benötigen Sie BDB, InnoDB oder Gemini-Tabellen.

Sie können innerhalb einer Datenbank Tabellen unterschiedlicher Typen verwenden und so das Tabellenformat Ihren Anforderungen anpassen. Dieses Konzept ist schlau, da Transaktionen nicht für jede Anwendung benötigt werden. Sicherheit und Komfort gehen natürlich zu Lasten von Geschwindigkeit. Innerhalb einer DB können Tabellen vom Typ MyISAM und InnoDB (ebenso wie *HEAP*, *ISAM*, und *BDB*) gemischt eingesetzt werden.

Es gibt verschiedene Möglichkeiten, Transaktionen zu verwalten. Die Tabellentreiber der BDB-, InnoDB- und Gemini-Tabellen verwenden jeweils unterschiedliche Transaktionsmodelle. Die Unterschiede betreffen in erster Linie die Art und Weise, wie Datensätze während einer Transaktion vor Änderungen durch andere Anwender geschützt werden.

Bei MyISAM-Tabellen kann nur die gesamte Tabelle durch *LOCK* geschützt werden. Bei BDB-Tabellen werden während einer Transaktion automatisch (also ohne ein explizites *LOCK*-Kommando) alle erforderlichen Datensätze gesperrt. Intern werden dabei allerdings ganze Speicherseiten gesperrt, weshalb es passieren kann, dass einige in der Nähe gespeicherte Datensätze während der Transaktion ebenfalls nicht mehr zugänglich sind.

Auch bei InnoDB- und Gemini-Tabellen werden die Datensätze automatisch gesperrt. Allerdings ist das Locking hier durch zusätzliche SQL-Kommandos genauer steuerbar als bei BDB-Tabellen. Ein weiterer Vorteil besteht darin, dass hier wirklich nur die betroffenen Datensätze gesperrt werden. Diese intelligenteste Form des Locking hat insbesondere den Vorteil, dass andere MySQL-Anwender so wenig wie möglich behindert werden.

Alle drei Tabellentreiber erkennen im Regelfall automatisch Deadlock-Situationen, bei denen einzelne Prozesse endlos aufeinander warten. In solchen Fällen wird bei dem Prozess, der den Deadlock ausgelöst hat, automatisch ein *ROLLBACK* ausgeführt.

Als zusätzliches Feature versprechen sowohl InnoDB als auch Gemini ein automatisches Crash-Recovery. Das bedeutet, dass alle Tabellen beim nächsten Start nach einem Stromausfall automatisch wiederhergestellt werden, wobei die Software alle bis zum Stromausfall vollständig ausgeführten Transaktionen berücksichtigt.



Locking-Mechanismen

- | | |
|----------|------------------------------------|
| ▪ MyISAM | Table-Locking |
| ▪ BDB | Page-Level-Locking - Speicherseite |
| ▪ Gemini | Row-Level-Locking – Datensatz |
| ▪ InnoDB | Row-Level-Locking - Datensatz |

Die Beschreibung jeder Tabelle wird in einer **.FRM*-Datei gespeichert, die in einem Verzeichnis liegt, die dem Namen der DB entspricht. Und wo sich dieses Verzeichnis befindet ist in der Optionsdatei *my.cnf* definiert.

	MyISAM	InnoDB
Eigenschaften	▪ stabil und schnell ▪ braucht wenig Speicherplatz ▪ unterstützt Table locking	▪ unterstützt Transaktionen und referentielle Integrität ▪ braucht mehr Speicherplatz ▪ unterstützt Row locking
Einsatz	▪ wenn Geschwindigkeit wichtig ▪ bei vielen Daten ▪ für effiziente Verwaltung	▪ wenn Sicherheit wichtig ist ▪ wenn viele Anwender gleichzeitig Daten ändern
Speicherung	Jede MyISAM-Tabelle wird in 3 Tabellen gespeichert: ▪ *.FRM (Tabellenbeschreibung), ▪ *.MYD (Daten), ▪ *.MYI (Indexe).	▪ *.FRM-Datei für jede Tabelle ▪ Daten in einer gemeinsamen Datei - der Tablespace-Datei ▪ Bei: innodb_file_per_table wird zusätzlich ein Tablespace pro Tabelle angelegt (*.ibd)

Tab. 11.1: Die beiden wichtigsten Tabellentypen von MySQL: MyISAM und InnoDB im Vergleich.

11.2 Transaktionen

Mit Transaktionen können mehrere SQL-Anweisungen in einem Block gekapselt werden, um diesen Block dann "sicher" auszuführen. Sicher bedeutet hier „ganz oder gar nicht“. Transaktionen erhöhen in vielen Fällen die Sicherheit und Konsistenz der Daten, erleichtern die Programmierung und erhöhen eventuell auch die Effizienz von Anwendungen. Wir verwenden im Folgenden den Tabellentyp InnoDB.



11.2.1 Wozu Transaktionen?

Dieser Abschnitt erklärt, warum Transaktionen wichtig sind und wie sie unter MySQL eingesetzt werden. Transaktionen stellen sicher, dass eine Gruppe von SQL-Kommandos, die mit *BEGIN* beginnt und mit *COMMIT* endet, entweder vollständig oder gar nicht ausgeführt wird. Selbst wenn während der Operation der Strom ausfällt oder der Rechner abstürzt, kann es nicht passieren, dass nur ein Teil der Kommandos ausgeführt wird.

Eine derartige Gruppierung mehrerer Kommandos ist beispielsweise dann notwendig, wenn Sie einen Betrag von Konto 1 auf Konto 2 abbuchen möchten. Es darf nicht passieren, dass die Abbuchung von Konto 1 durchgeführt wird, nicht aber die Buchung auf Konto 2.

Transaktionen stellen zugleich sicher, dass die Daten nicht quasi gleichzeitig von anderen Anwendern verändert werden. Das ist beispielsweise erforderlich, um die referenzielle Integrität einer Datenbank sicherzustellen. Damit ist gemeint, dass verknüpfte Tabellen nur auf existierende Datensätze und nicht ins Leere verweisen. Nehmen Sie etwa an, Sie wollen einen Datensatz mit *id=x* aus Tabelle A löschen. Zuerst prüfen Sie, ob es keinen Datensatz in Tabelle B gibt, der auf den zu löschenden Datensatz verweist:

```
SELECT COUNT(*) FROM tableB WHERE idA=x
```

Nur wenn diese Abfrage 0 als Ergebnis liefert, löschen Sie den Datensatz in Tabelle A:

```
DELETE FROM tableA WHERE id=x
```

Wenn Sie diese beiden Kommandos nicht durch eine Transaktion miteinander verbinden, kann es aber passieren, dass ein anderer Client genau zwischen diesen beiden Kommandos einen neuen Datensatz in die Tabelle B einfügt:

```
INSERT INTO tableB (... , idA) VALUES (... , x)
```

Damit gäbe es einen Datensatz in Tabelle B, der auf einen nicht mehr existenten Datensatz in Tabelle A verweist - das ist wie ein Link auf eine nicht mehr bestehende Webseite.

Ohne Transaktionen können Sie einen Schutz gegen derartige Probleme nur durch *LOCK*-Kommandos erreichen. Damit wird aber kurzzeitig eine ganze Tabelle für alle Clients blockiert, was sehr ineffizient ist. Bei den transaktionstauglichen Tabellentypen werden nur die Datensätze blockiert, die tatsächlich verändert werden sollen. Diese intelligentere Form des Zugriffsschutzes wird als **Row Level Locking** bezeichnet.

Transaktionen erhöhen schließlich auch den Programmierkomfort. Eine Transaktion kann jederzeit abgebrochen werden, was die Fehlerabsicherung sehr vereinfacht. Wenn also nach mehreren SQL-Kommandos ein Fehler auftritt, können Sie alle bisher im Rahmen einer Transaktion durchgeführten Kommandos durch ein einfaches *ROLLBACK* widerrufen.

Ob Sie nun tatsächlich Transaktionen benötigen oder nicht, hängt sehr stark davon ab, welche Anwendungen Sie mit MySQL entwickeln. Ein einfaches Diskussionsforum im Internet funktioniert auch ohne Transaktionen zuverlässig. Wenn Sie dagegen ein Buchhaltungsprogramm, eine komplexe Lagerverwaltung oder ähnliche Anwendungen entwickeln oder wenn die von Ihnen verwalteten Daten relativ sensitiv sind - also etwa medizinische oder finanzielle Daten -, dann sind Transaktionen und eine entsprechende Programmierung unabdingbar. Das bisherige Fehlen von Transaktionen in MySQL ist ein Grund dafür, warum MySQL für derartige Anwendungen bis jetzt kaum zum Einsatz kam.



11.2.2 **Nachteile von Transaktionen**

Normale Tabellenoperationen sind in Tabellen mit Transaktionsunterstützung im Regelfall langsamer, weil der Verwaltungsaufwand größer ist. Genau das ist auch der Grund, warum sich die MySQL-Entwickler lange geweigert haben, Transaktionen zu unterstützen.

Transaktionen stellen schließlich größere Anforderungen an den Programmierer: Zum einen muss er sich mit dem Transaktionsmodell des jeweiligen Tabellentreibers auseinandersetzen und die SQL-Kommandos zur Steuerung kennen und verstehen lernen. Zum anderen ist es nötig, den Ablauf besser als bisher abzusichern: Aufgrund von Transaktionen kann es dazu kommen, dass SQL-Kommandos momentan nicht ausgeführt werden können, weil andere Anwender die Daten gerade ändern. Wenn es dabei zu Verzögerungen kommt, wird Ihr SQL-Kommando nach einer bestimmten Zeit abgebrochen.

11.2.3 **notwendige Anpassungen in der my.cnf/my.ini für InnoDB-Tabellen**

InnoDB-Tabellen werden nicht wie alle anderen Tabellentypen in einfachen Dateien gespeichert. Stattdessen dient eine zentrale Datei, die als Tablespace bezeichnet wird, als virtueller Speicher für alle InnoDB-Tabellen und -Indizes. Die Standareinstellungen in der MySQL-Optionsdatei sind für die Erstellung von MyISAM-Dateien eingerichtet und InnoDB-Tabellen sind deaktiviert. Um das zu ändern, müssen die Einstellungen wie folgt vorgenommen werden:

```
#skip-innodb
# Uncomment the following if you are using InnoDB tables
innodb_data_home_dir = "C:/MySQL/data"
innodb_data_file_path = ibdata_tbz:10M:autoextend
innodb_log_group_home_dir = "C://MySQL/data"
innodb_log_arch_dir = "C:/MySQL/data"
# You can set .._buffer_pool_size up to 50 - 80 %
# of RAM but beware of setting memory usage too high
set-variable = innodb_buffer_pool_size=16M
set-variable = innodb_additional_mem_pool_size=2M
# Set .._log_file_size to 25 % of buffer pool size
set-variable = innodb_log_file_size=5M
set-variable = innodb_log_buffer_size=8M
innodb_flush_log_at_trx_commit=1
set-variable = innodb_lock_wait_timeout=50
# innodb_file_per_table
```

Die Einstellungen werden natürlich erst nach dem Start des DB-Servers übernommen. Danach sollte es aber möglich sein, den Tabellentyp InnoDB auszuwählen.

InnoDB-Tabelle erzeugen:

Um eine InnoDB-Tabelle zu erstellen, wird dies im *CREATE TABLE*-Befehl angegeben. Im phpMyAdmin können Sie diese Angabe in einem Auswahlfenster angeben. Das folgende Beispiel erstellt eine neue DB und die InnoDB-Tabelle *konten*:



```
mysql> CREATE DATABASE innotest;
mysql> USE innotest
mysql> CREATE TABLE konten (k_id INT AUTO_INCREMENT,
                             saldo DECIMAL(8,2),
                             PRIMARY KEY (k_id)) ENGINE = InnoDB;
```

Sie können auch bestehende Tabellen nachträglich in InnoDB-Tabellen umwandeln. Das geht entweder mit dem SQL-Befehl `ALTER TABLE...` oder unter Operationen im phpMyAdmin.

```
ALTER TABLE tblname ENGINE = InnoDB
```

⚠ Ändern Sie auf keinen Fall den Tabellentyp der Tabellen der Datenbank *mysql*! Diese Tabellen mit MySQL-internen Verwaltungsinformationen (Benutzer- und Zugriffsrechte) müssen immer im MyISAM-Format bleiben!

Wenn Sie feststellen möchten, wie viel Speicherplatz innerhalb des *Tablespace* noch frei ist, führen Sie einfach für eine beliebige Datenbank mit InnoDB-Tabellen das Kommando `SHOW TABLE STATUS` aus. Der freie Platz (der insgesamt, also für alle InnoDB-Tabellen zusammen gilt), wird dann in der Kommentarspalte angezeigt.

Eine Verkleinerung des Tablespace ist nicht vorgesehen. Der einzige Weg besteht darin, die Tabellen mit `mysqldump` und dem Parameter `--all-databases` zu exportieren, einen verkleinerten Tablespace neu zu erzeugen und die Tabellen dorthin wieder zu importieren: `>mysqldump -u root -p --all-databases > alldb.sql`

11.2.4 **Transaktionen und Logging**

MySQL kann Änderungen in der Datenbank wahlweise im Textmodus oder binär protokollieren. Die resultierenden Logging-Dateien können beispielsweise für inkrementelle Backups oder zur Replikation mehrerer Datenbank-Server verwendet werden. Dazu später mehr.

Wenn Sie mit Transaktionen arbeiten, müssen Sie unbedingt binäres Logging verwenden, weil nur diese Variante Transaktionen korrekt behandelt. Wenn Sie das Logging dagegen im Textmodus durchführen, werden auch solche Kommandos protokolliert, die durch `ROLLBACK` widerrufen werden. Binäres Logging wird durch die Option `log-bin` in der Optionsdatei `my.cnf` aktiviert.

11.2.5 **BEGIN, COMMIT und ROLLBACK**

Indem Sie eine Tabelle vom Typ BDB, InnoDB oder Gemini erzeugen, haben Sie noch nichts gewonnen. Damit Sie den Vorteil von Transaktionen nutzen können, müssen Sie diese mit `BEGIN` einleiten und mit `COMMIT` beenden. `COMMIT` führt alle seit `BEGIN` angegebenen SQL-Kommandos tatsächlich aus.

Statt `COMMIT` auszuführen, können Sie die gesamte Transaktion aber auch mit `ROLLBACK` widerrufen. `ROLLBACK` wird automatisch ausgeführt, falls es zu einem Verbindungsabbruch kommt.

Wie weit offene Transaktionen den Lesezugriff auf eine Tabelle und deren Veränderung durch andere Clients blockieren, hängt von der Implementierung der Transaktionsunterstützung ab.



11.2.6 **Der Autocommit-Modus**

MySQL befindet sich normalerweise im Autocommit-Modus (*AUTOCOMMIT=1*). Alle SQL-Kommandos, die nicht durch ein vorheriges *BEGIN* als Transaktion gekennzeichnet sind, werden daher sofort ausgeführt.

Wenn Sie das SQL-Kommando *SET AUTOCOMMIT=0* ausführen, wird der Autocommit-Modus ausgeschaltet. Das bedeutet, dass alle SQL-Kommandos als eine große Transaktion gelten. Die Transaktion wird durch *COMMIT* oder *ROLLBACK* abgeschlossen. Damit beginnt automatisch eine neue Transaktion. Sie müssen also kein *BEGIN* mehr ausführen. Allein diese Bequemlichkeit ist oft Grund genug, um mit *SET AUTOCOMMIT=0* zu arbeiten.

Eine wichtige Konsequenz von *AUTOCOMMIT=0* besteht darin, dass bei einem Verbindungsabbruch - egal ob er beabsichtigt oder unbeabsichtigt eingetreten ist - alle SQL-Kommandos, die nicht durch *COMMIT* bestätigt sind, widerrufen werden.

Beachten Sie auch, dass durch *AUTOCOMMIT=0* sehr lange Transaktionen entstehen, wenn Sie nicht regelmäßig *COMMIT* oder *ROLLBACK* ausführen. Manche Tabellentreiber kommen mit derart langen Transaktionen schlecht zurecht, das heißt es können Locking-Probleme auftreten, die die Effizienz beeinträchtigen. Aufgrund seiner Architektur kommt der InnoDB-Tabellentreiber mit derartigen Transaktionen überdurchschnittlich gut zurecht.

11.2.7 **InnoDB - möglichst nah an Oracle**

Der InnoDB-Tabellentreiber ist eine Entwicklung der finnischen Firma Innobase und seit der MySQL-Version 3.23.34a erhältlich. Der Autor des Tabellentreibers legt großen Wert darauf, viele Merkmale des Tabellentreibers möglichst ähnlich wie bei Oracle zu lösen.

11.2.8 **Locking-Verhalten von InnoDB**

Der InnoDB-Tabellentreiber kümmert sich selbständig um alle notwendigen Locking-Operationen. Beachten Sie aber, dass Sie das Kommando *LOCK TABLE* nicht ausführen dürfen. Alle durch *INSERT*, *UPDATE*, *DELETE* veränderten Datensätze werden automatisch bis zum Ende der Transaktion durch einen so genannten Exklusive Lock gesperrt. Das heißt: Andere Transaktionen können diese Datensätze nicht verändern.

Eine Besonderheit von InnoDB besteht darin, dass gewöhnliche *SELECT*-Kommandos trotz gesperrter Datensätze immer sofort ausgeführt werden. Allerdings ignorieren die zurückgegebenen Resultate noch laufende Transaktionen anderer Clients, liefern also eventuell veraltete Daten (siehe Verbindung B zum Zeitpunkt 1 im Kasten "Transaktionsbeispiel").

Wenn dieses Default-Verhalten für Ihre Anwendung nicht optimal ist, bieten zwei Erweiterungen des *SELECT*-Kommandos die Möglichkeit, das Locking-Verhalten gezielt zu steuern: Zum einen können Sie einzelne Datensätze auch ohne eine tatsächliche Veränderung exklusiv sperren. Das empfiehlt sich, wenn Sie die Datensätze zuerst lesen und anschließend ändern möchten. Dazu verwenden Sie das *SELECT*-Kommando mit der Erweiterung *FOR UPDATE*:



```
BEGIN
SELECT * FROM table WHERE x>10 FOR UPDATE
-- die ausgewählten Datensätze
-- sind jetzt exklusiv gesperrt
COMMIT -- oder ROLLBACK
```

Sie können auch mit *SELECT ... LOCK IN SHARE MODE* explizit darauf warten, bis alle noch offenen Transaktionen abgeschlossen sind - genau genommen wartet *SELECT*, bis alle Exclusive Locks aufgelöst sind.

Gleichzeitig werden die von *SELECT* gefundenen Datensätze nun mit einem so genannten Shared Lock gesperrt. Ein Shared Lock ist nicht ganz so stark wie ein Exclusive Lock. Die Daten können von anderen Anwendern ebenfalls nicht verändert werden, lassen sich aber mit *SELECT ... LOCK IN SHARE MODE* lesen.

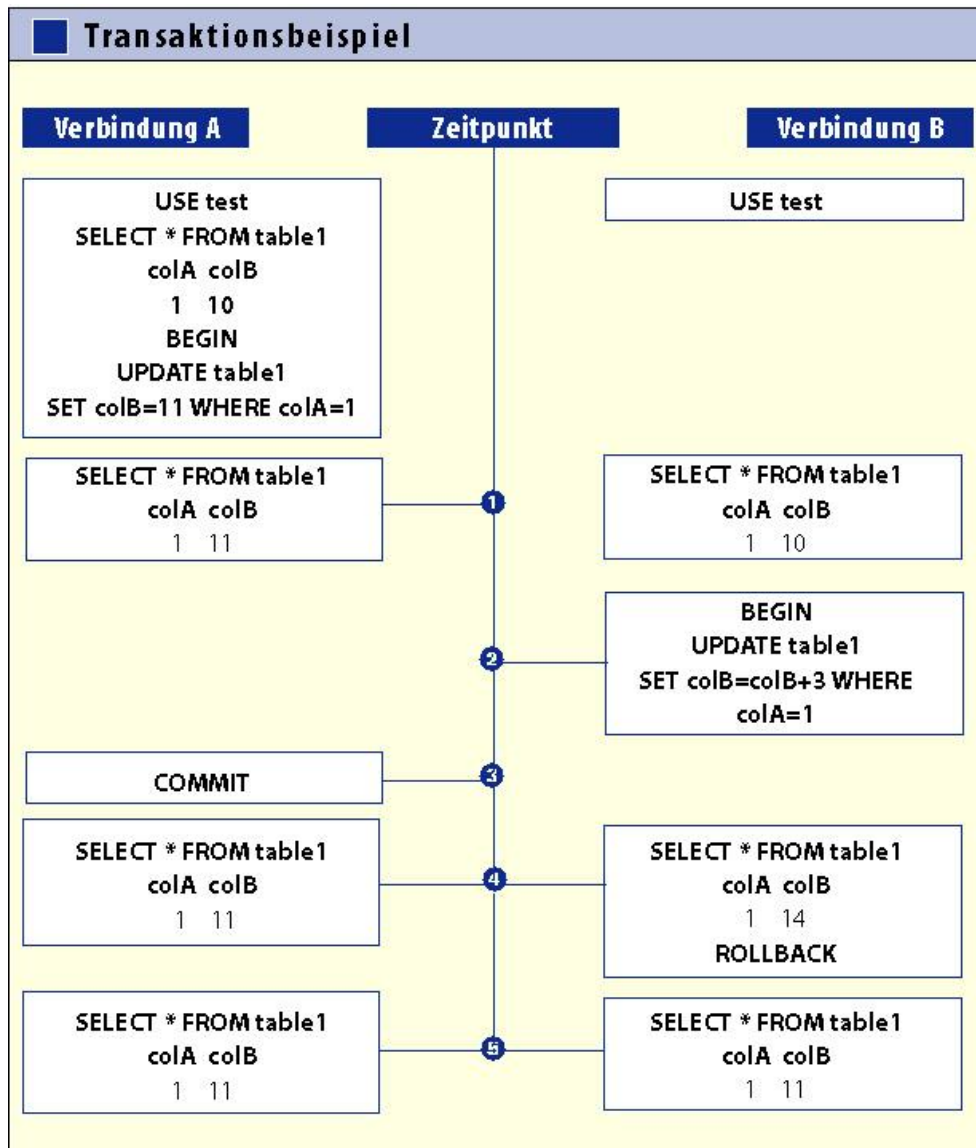
```
BEGIN
SELECT * FROM table
WHERE x>10 LOCK IN SHARE MODE
-- die ausgewählten Datensätze sind
-- jetzt durch shared locks gesperrt
COMMIT -- oder ROLLBACK
```

SELECT ...LOCK IN SHARE MODE sollte dann eingesetzt werden, wenn Sie die Datensätze zwar selbst nicht verändern, aber sicher sein möchten, dass die Daten auch von anderen Anwendern nicht geändert werden.

Die maximale Wartezeit auf die Freigabe gesperrter Datensätze wird in der MySQL-Konfigurationsdatei durch *set-variable=innodb_lock_wait_timeout=n* eingestellt (in Sekunden). Verstreicht diese Zeit, wird die gesamte Transaktion durch *ROLLBACK* abgebrochen.



11.2.9 Transaktionen ausprobieren



Um Transaktionen ausprobieren zu können, müssen Sie zwei Verbindungen zur Datenbank herstellen. Am einfachsten führen Sie dazu den Monitor *mysql* in zwei Fenstern aus. Anschließend führen Sie die im Kasten "Transaktionsbeispiel" dargestellten Kommandos mal in dem einen, mal im anderen Fenster aus.

Das Beispiel geht davon aus, dass es in der Datenbank *test* die InnoDB- Tabelle *table1* gibt. In dieser Tabelle muss sich mindestens ein Datensatz befinden. Natürlich werden Sie in der Praxis mehr Datensätze haben, aber zur Demonstration des Mechanismus reicht ein Datensatz aus.

```
USE test
CREATE TABLE table1
(
  colA INT AUTO_INCREMENT,
  colB INT,
  PRIMARY KEY (colA)) TYPE=InnoDB
INSERT INTO table1 (colB) VALUES (10)
```



Zum Zeitpunkt 1 sieht der Inhalt von *table1* aus Sicht der beiden Verbindungen unterschiedlich aus. Für Verbindung A gilt für den Datensatz mit *colA=1* bereits *colB=11*. Da diese Transaktion aber noch nicht tatsächlich ausgeführt ist, sieht Verbindung B für denselben Datensatz *colB=10*. Zum Zeitpunkt 2 beginnt B eine Transaktion; *colB* des Datensatzes mit *colA=1* soll um drei vergrößert werden. Der InnoDB-Tabellentreiber erkennt, dass er dieses Kommando zur Zeit nicht ausführen kann, und blockiert B, die jetzt darauf wartet, dass A die Transaktion beendet.

Zum Zeitpunkt 3 schließt A die Transaktion mit *COMMIT*. Damit enthält *colB* definitiv den Wert 11. Jetzt kann auch das *UPDATE*-Kommando von B abgeschlossen werden. Zum Zeitpunkt 4 sieht A *colB=11*. Für B sieht es so aus, als hätte *colB* bereits den Wert 14. Nun widerruft B die Transaktion. Damit sehen A und B zum Zeitpunkt 5 beide den tatsächlich gespeicherten Wert *colB=11*.

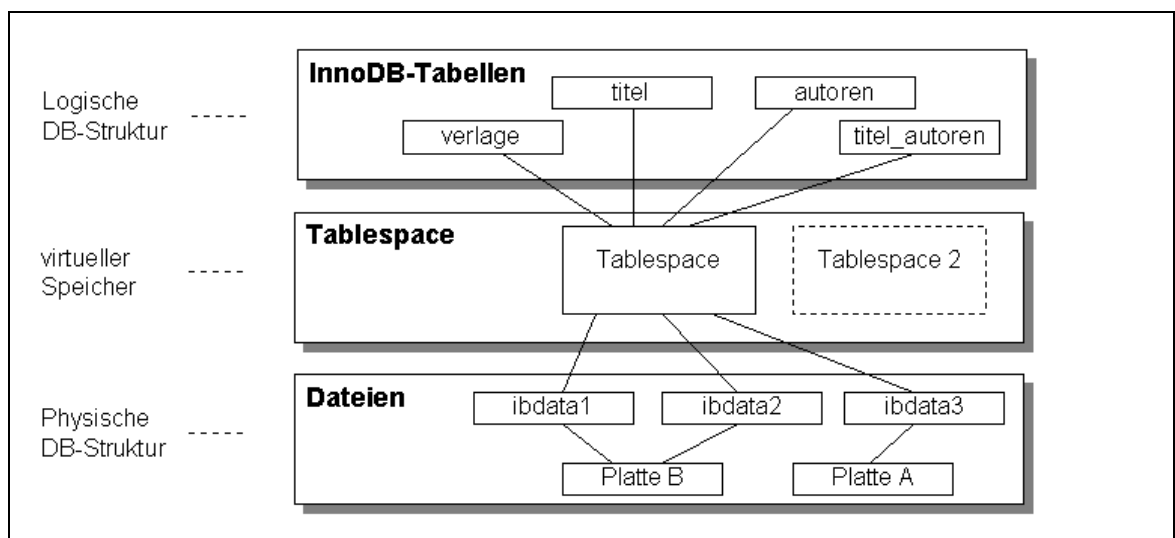


Bild 11.1: Tablespace als Schicht zwischen DB-Tabellen und DB-Dateien

Standardmässig wird beim ersten Start von MySQL die Datei *ibdata1* mit einer Größe von 10 MByte erzeugt. Sobald die darin gespeicherten InnoDB-Tabellen mehr Platz beanspruchen, wird die Datei in 8-MByte-Schritten vergrößert (*autoextend*). Zusätzlich werden auch die Logging-Dateien *ib_logfile0*, *-1* und *ib_arch_log_000000000* erzeugt. Alle vier Dateien liegen im binären Format vor und dürfen nicht verändert werden.

11.3 ACID

Die Abkürzung **ACID** steht für die notwendigen Eigenschaften, um Transaktionen auf Datenbankmanagementsystemen (DBMS) einsetzen zu können. Es steht für **A**tomarität (atomicity), **K**onsistenz (consistency), **I**soliertheit (isolation) und **D**auerhaftigkeit (durability). Im Deutschen spricht man gelegentlich auch von AKID.

11.3.1 Atomarität

Ein Block von SQL-Anweisungen (eine oder mehrere) wird entweder ganz oder gar nicht ausgeführt: Das Datenbanksystem behandelt diese Anweisungen wie eine einzelne, unteilbare (daher atomare) Anweisungen.



Gibt es bei einem Statement technische Probleme oder tritt ein definierter Zustand ein, der einen Abbruch erfordert (z. B. Kontoüberziehung), werden auch die bereits durchgeführten Operationen nicht auf der Datenbank wirksam.

11.3.2 **Konsistenz**

Nach Abschluss der Transaktion befindet sich die Datenbank in einem konsistenten Zustand. Das bedeutet, dass Widerspruchsfreiheit der Daten gewährleistet sein muss. Das gilt für alle definierten Integritätsbedingungen genauso, wie für die Schlüssel- und Fremdschlüsselverknüpfungen.

11.3.3 **Isoliertheit**

Die Ausführungen verschiedener Datenbankmanipulationen dürfen sich nicht gegenseitig beeinflussen. Mittels Sperrkonzepten oder Timestamps wird sichergestellt, dass durch eine Transaktion verwendete Daten nicht vor Beendigung dieser Transaktion verändert werden. Die Sperrungen müssen so kurz und begrenzt wie möglich gehalten werden, da sie die Performance der anderen Operationen beeinflusst.

11.3.4 **Dauerhaftigkeit**

Nach Abschluss einer Transaktion müssen die Manipulationen dauerhaft in der Datenbank gespeichert sein. Der Pufferpool speichert häufig verwendete Teile einer Datenbank im Arbeitsspeicher. Diese müssen aber nach Abschluss auf der Festplatte gespeichert werden.



12 Stored Procedures und Funktionen

TBD



13 Index

A	I	P	Row-Locking20
ACID-Regeln27	InnoDB..... 20	Page-Level-Locking 20	T
autoextend27	M	R	Table locking.....20
	MyISAM 20	Row-Level-Locking 20	Table-Locking20